
This State Looks Like That: Self-Interpretable Reinforcement Learning Agents using Prototype Soft Actor-Critic

Andrea Marzo^{1 2 3} Alessio Ragno² Roberto Capobianco⁴

Abstract

Reinforcement learning (RL) has achieved remarkable success across complex decision-making tasks, especially with the advent of deep neural networks. However, the resulting models are often opaque, making their deployment in safety-critical domains challenging. Explainable AI aims to address this issue, but most specific efforts for deep RL remain limited either to post-hoc explanation methods or to imitation learning and distillation procedures. These latter approaches rely on pre-trained black-box agents and are typically restricted to environments with discrete action spaces, limiting their scalability and interpretability. In this paper, we introduce ProtoSAC, a novel deep RL architecture that integrates a prototype-based actor into the Soft Actor-Critic (SAC) algorithm, enabling intrinsic interpretability in continuous action spaces. Our method learns a set of prototypes that represent interpretable state clusters, each associated with a Gaussian action distribution. Actions are generated as a similarity-weighted mixture over these prototypes, providing more inspectable and decomposable decision-making without sacrificing performance compared to standard SAC. We evaluate ProtoSAC on continuous control environments and show that it matches the performance of the original SAC while offering enhanced interpretability.

1. Introduction

Deep reinforcement learning (DRL) models reach remarkable capabilities in solving complex sequential decision-making tasks (Silver et al., 2016), achieving super-human performance in areas such as game playing, robotics (Kober et al., 2013), and autonomous control (Kiumarsi et al., 2017). The incorporation of deep neural networks into reinforcement learning (RL) frameworks expands their scope and efficacy, allowing them to handle high-dimensional input spaces. However, despite these significant advancements, DRL models suffer from a crucial limitation: their decision-making processes often remain opaque and unintelligible to human observers, preventing their adoption in safety-critical and trust-dependent applications.

To address this limitation, eXplainable Artificial Intelligence (XAI) emerges as a critical research area aiming to enhance the transparency and interpretability of otherwise black-box models. Within XAI, the subfield of explainable DRL specifically focuses on elucidating the decision-making processes of RL agents by either post-hoc interpretability techniques or designing self-interpretable agents from the ground up. However, the majority of existing explainable methodologies predominantly focus on classical RL problems rather than DRL settings, and often rely heavily on post-hoc explanations that attempt to interpret already trained, opaque models. While post-hoc methods can offer insights into agent behavior, they are generally less faithful (Rudin, 2019), as most of them (Bastani et al., 2018; Verma et al., 2018; Frosst & Hinton, 2018) do not reflect the true internal decision-making process of the model but instead approximate it. In contrast, self-explainable agents are designed to make their reasoning more directly inspectable, offering higher fidelity in the explanations and stronger guarantees of alignment between the model’s behavior and the provided interpretations.

Few studies explore the concept of self-interpretable DRL agents. Among these, our work is closely related to the ones of Kenny et al. (2023) and Borzillo et al. (2023). They propose two architectures, namely PW-Net and Shared PW-Net, which employ prototypical classifiers to provide ex-

¹Université de Rennes, CNRS, Inria, Insa Rennes, IRISA, FR-35000, Rennes, France ²EPITA Research Laboratory (LRE), FR-69621, Le Kremlin-Bicetre, France ³Department of Computer, Control and Management Engineering (DIAG), Sapienza University, IT-00185, Rome, Italy ⁴Sony AI, CH-8952, Zurich, Switzerland. Correspondence to: Andrea Marzo <andrea.marzo@irisa.fr>, Alessio Ragno <alessio.ragno@epita.fr>, Roberto Capobianco <roberto.capobianco@sony.com>.

planations for RL agents through distillation and imitation learning. These approaches leverage prototype-based interpretability, a class of techniques that use prototypes (i.e., representative parts or full instances of the training data) to ground the inference process. However, both PW-Net and Shared PW-Net rely on the pre-training of a conventional black-box agent, which is subsequently distilled into an interpretable agent. This limits the learning capabilities to what the black-box model has already learned.

In this paper, we propose a novel self-interpretable DRL model designed specifically to address the limitations identified in prior works in continuous action-space scenarios. Our approach integrates a prototype-based component within the Soft Actor-Critic (SAC) algorithm (Haarnoja et al., 2018). Specifically, we design an actor network that integrates a prototypical layer to enable prototype-based decision-making. This layer directly compares input states against a set of learned prototypes and determines the action based on the resulting similarity scores. Each prototype corresponds to a specific action distribution characterized by mean and standard deviation parameters. The final action is produced as a similarity-weighted combination of these prototypes. By embedding interpretability directly within the policy learning framework, our approach provides intrinsic interpretability from scratch¹, avoiding the need for distillation of pre-trained agents, and fully supports continuous action spaces.

Overall, the main contributions of our work are as follows: 1) we propose *ProtoSAC*, the first self-interpretable reinforcement learning agent that integrates a prototype-based actor directly into the SAC framework for continuous action spaces; 2) we design a novel architecture where the actor outputs are defined as Gaussian distributions through a similarity-weighted combination over learned prototypes, providing decomposable and prototype-based decision-making; 3) we introduce a prototype update mechanism and additional regularization losses that enhance coverage, diversity, and interpretability of the prototypes throughout training; 4) we conduct an experimental evaluation across benchmark environments to assess *ProtoSAC*’s performance and interpretability, including ablation studies on loss terms and prototype quantity; 5) we provide an open-source implementation of our proposed method and experimental setup².

The remainder of this work is structured as follows: we

¹Different meanings have been attributed to interpretability in the literature. Here, we refer to “interpretable” and “transparent” in the sense common to prototype-based methods: providing a decomposable decision process and prototype-level evidence for inspection, rather than full transparency of the raw input-to-decision mapping.

²Public GitHub repository: <https://github.com/KRLGroup/PrototypeSAC>

begin by reviewing the current literature on explainable artificial intelligence and interpretable reinforcement learning; we then formalize the theoretical background including SAC and prototype-based models; subsequently, we introduce the architecture and training procedure of *ProtoSAC*; we follow with an experimental evaluation on continuous action-space environments; finally, we conclude by summarizing our findings and discussing future research directions.

2. Related Work

In this section, we review related state-of-the-art methodologies, first discussing the broader field of XAI and subsequently focusing on its application to DRL.

Explainable Artificial Intelligence. XAI is a research field focusing on making the decisions of AI models understandable to humans. XAI methods can be primarily categorized into two classes: post-hoc methods and self-explainable models (Došilović et al., 2018). Post-hoc methods aim at providing explanations for already trained black-box models, typically by analyzing model predictions after training has been completed. In contrast, self-explainable models focus on designing architectures that are interpretable by construction, embedding transparency into the model structure itself and thus enabling the generation of explanations simultaneously with predictions. Our work specifically targets the development of a self-explainable model within the domain of DRL.

In the literature of self-explainable approaches, two primary directions are explored: concept-based models (Koh et al., 2020; Wang et al., 2023a) and prototype-based models (Chen et al., 2019; Wang et al., 2023b). Concept-based models attempt to identify and leverage meaningful human-interpretable concepts present in the input data. The predictions of these models are subsequently derived based on the presence or activation of such concepts. However, a significant limitation of these methods is the necessity of pre-labeled datasets with concept annotations, a requirement that is challenging to fulfill in many scenarios. In contrast, prototype-based models do not require pre-labeled concepts, as they automatically learn exemplary instances, known as prototypes, directly during the training phase. Predictions are then made based on the similarity between the input and these learned prototypes, which provides prototype-level signals that aid inspection of the policy behavior.

Explainable Deep Reinforcement Learning. Within the context of DRL, explanations can be provided at different stages of the decision-making process. Some methods focus on clarifying the reasoning behind the agent’s actions (actor explanations), others interpret environment dynamics (world explanations), and some approaches explicitly explain how the reward signals shape the agent’s behavior (reward ex-

planations) (Qing et al., 2023; Milani et al., 2024; Glanois et al., 2024). In particular, we focus on those approaches that aim at generating an interpretable actor, that is able to provide explanations together with decisions. Most of these approaches proceed by either distilling pre-trained models or through an imitation learning (IL) approach. Many approaches for instance, propose to approximate the policy and the Q-value functions using decision trees, programmatic policies or even logic (Bastani et al., 2018; Verma et al., 2018; Frosst & Hinton, 2018). However, these approaches increase the interpretability by approximating deep neural networks with simpler models. Delfosse et al. (2024), instead propose a concept-based approach that grounds the input state to human-designed concepts and learns logic rules over these concepts. Despite the fact that in this case the concepts are actually learned using deep neural networks, this work requires a manual labeling of the concepts, which is not actually feasible in all cases. A novel research direction involves prototype-based approaches. These methods originate from architectures initially developed for image classification, such as ProtoPNet (Chen et al., 2019), where a learnable weight matrix stores representative samples, called prototypes, from the training set. The model computes its output based on the similarity between the input and these prototypes. Ragodos et al. (2022) present one of the first applications of prototype-based methods in explainable DRL through ProtoX. However, their method is post-hoc: it explains a black-box agent rather than producing an inherently interpretable one.

PWNet (Kenny et al., 2023) represents the first attempt to build self-explainable DRL agents using prototypes. Kenny et al. (2023) adopt an IL framework, transforming the reinforcement learning task into a supervised one, which enables the use of models like ProtoPNet. In PWNet, decisions are inferred by comparing the input state to a set of prototypes. A key limitation of this approach is its reliance on manually selected prototypes, which restricts its applicability in scenarios lacking prior domain knowledge. To address this, Borzillo et al. (2023) propose Shared-PWNet, which automates the prototype selection process. While promising, this method still depends on IL. In contrast, our approach, ProtoSAC, integrates the prototypical framework directly into the Soft Actor-Critic (SAC) algorithm. This allows us to constrain the agent’s policy toward interpretable behaviors without requiring IL or manual prototype selection. To clarify the distinctions between these prototype-based approaches, Table 1 summarizes their main characteristics. Finally, it is worth mentioning that although prototypes have been explored in continuous action spaces (Yarats et al., 2021), they have not been explored for transparency. Unlike ProtoRL, which leverages them for exploration, our approach employs prototypes for decomposable reasoning to enhance inspectability.

Table 1. Comparison of prototype-based approaches for explainable DRL.

Approach	Self-Interpretable	Learning	Prototype Selection
ProtoX	No	-	Automatic
PW-Net	Yes	IL	Manual
Shared-PW-Net	Yes	IL	Automatic
ProtoSAC (Ours)	Yes	RL	Automatic

3. Background

In this section, we define the mathematical basis for the understanding of our proposal. We start by introducing the reinforcement learning setting; we then follow with the description of the SAC algorithm; finally, we present the basis of prototype-based approaches.

Reinforcement Learning is a subfield of machine learning where an agent learns the correct behavior by interacting with the environment and leveraging reward signals to improve its decision-making over time. Formally, RL problems are often modeled as Markov Decision Processes (MDPs), defined by the tuple (S, A, T, r, γ) where S is the set of states, A is the set of actions, $T(s', s, a)$ is the transition probability, $r(s, a)$ is the reward function, and $\gamma[0, 1]$ is the discount factor. At each time step, the agent observes a state $s \in S$, selects an action $a \in A$ according to a policy $\pi(a|s)$, receives a scalar reward $r(s, a)$, and transitions to the next state s' . The goal of the agent is to learn a policy that maximizes the expected discounted return G_t , defined as:

$$G_t = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]. \quad (1)$$

To achieve this objective, it is essential to estimate the long-term value of selecting specific actions in particular states. This is captured by the *Q-function* (or *action-value function*), which represents the expected return starting from a state s , taking an action a , and subsequently following a given policy π . Formally, the Q-function is defined as:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \middle| s_0 = s, a_0 = a \right]. \quad (2)$$

Modern RL algorithms often rely on function approximation (typically neural networks) to represent policies and value functions, enabling scalability to high-dimensional state and action spaces. Among these, *actor-critic methods* are particularly effective. In this setting, the *actor* selects actions based on a parameterized policy $\pi_\theta(a|s)$, while the *critic* estimates the value function, typically the action-value function $Q^\pi(s, a)$. In the remainder of this section, we go into the details of SAC, which is the algorithm on which we build upon to design our approach.

Soft Actor-Critic is an actor-critic algorithm designed for continuous action spaces (Haarnoja et al., 2018). It is based

on the maximum entropy RL framework, which aims to not only maximize expected cumulative rewards but also encourages exploration by maximizing the entropy of the policy. The objective function in maximum entropy RL is defined as:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))], \quad (3)$$

where ρ_π is the state-action distribution induced by the policy π , $\mathcal{H}(\pi(\cdot|s_t)) = -\mathbb{E}_{a_t \sim \pi} [\log \pi(a_t|s_t)]$ is the entropy of the policy at state s_t , and α is a temperature parameter that controls the trade-off between reward maximization and entropy. SAC maintains three types of neural networks: a stochastic policy network $\pi_\theta(a|s)$ (actor), two Q-value networks $Q_{\phi_1}(s, a)$ and $Q_{\phi_2}(s, a)$ (critics), and corresponding target networks for stability, which are exponential moving averages of the critics. The Q-functions are trained to minimize the soft Bellman residual. The target value is computed as:

$$y = r + \gamma \mathbb{E}_{a' \sim \pi(\cdot|s')} \left[\min_{i=1,2} Q_{\bar{\phi}_i}(s', a') - \alpha \log \pi_\theta(a'|s') \right]. \quad (4)$$

The loss for each critic is then:

$$\mathcal{L}_Q(\phi_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(Q_{\phi_i}(s, a) - y)^2 \right], \quad i = 1, 2. \quad (5)$$

The policy network is updated by minimizing the following loss:

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\theta} \left[\alpha \log \pi_\theta(a|s) - \min_{i=1,2} Q_{\phi_i}(s, a) \right]. \quad (6)$$

While the critic models are only used at training time, the policy model is the one that is actually used at prediction time to execute actions. For this reason, in this paper, we propose to substitute the policy model to make its decision process more transparent through the use of prototypes.

Prototype-Based Methods attempt to explain a model's decisions by measuring the similarity between a given input and a set of representative examples, known as prototypes. As a result, these methods are usually built by extending a pre-trained agent with an additional explainability network. This network typically defines N distinct prototypes organized in a matrix $P \in \mathbb{R}^{N \times H}$, where H_P is the prototype hidden dimension. The architecture is then composed of three interconnected components that transform raw input states into interpretable action predictions: an *encoder*, a *prototype layer*, and a final *prediction layer*. First, the *encoder* $f_{enc}(s)$ maps the observed state s to a latent representation z :

$$z = f_{enc}(s). \quad (7)$$

The resulting representation is then compared against all the prototypes $p_i \in P$ of the *prototype layer*, thus computing

a similarity score for each of them. This score measures how closely the projected latent representation aligns with the prototypes, providing a basis for interpretable decision-making:

$$\text{sim}(z, p_i) = \log \left(\frac{\|z - p_i\|_2 + 1}{\|z - p_i\|_2 + \epsilon} \right) \quad i \in [1, \dots, N]. \quad (8)$$

Finally, these similarity scores are aggregated through a fully connected *prediction layer* to determine the final action, with weights $W \in \mathbb{R}^{N \times O}$, where O is the action dimension. This layer combines the prototype matching results to produce a set of action scores, from which the final action is selected:

$$a_j = \sum_{i=1}^N W_{i,j} \text{sim}(z, p_i) \quad j \in [1, \dots, O]. \quad (9)$$

4. ProtoSAC

We introduce *ProtoSAC*, a modified version of the SAC algorithm that integrates a prototype-based decision mechanism directly into the actor network. This design enables explainable decision-making during both training and deployment. We start by illustrating the general architecture, and then we proceed by presenting the adopted training procedure. In Figure 1 we depict an overview of the architecture, and in the remainder of this section we go over the details of the various components.

ProtoActor. *ProtoSAC* retains the original SAC structure (consisting of an actor, two critics, and a replay buffer) but replaces the standard actor with a prototype-based actor, referred to as the ProtoActor. This actor generates actions through a prototype-based representation, while the critic networks remain unchanged and operate directly on the resulting actions, thereby maintaining the original learning dynamics of SAC. Inspired by existing prototype-based methods, the input state is first passed through a state encoder f_s which maps the raw state into a latent representation z_s . This latent state is then fed into the ProtoActor. In order to produce continuous outputs, we pair the prototype matrix P with three additional elements: an importance weight $\tau \in \mathbb{R}_{>0}^N$, a mean $\mu \in \mathbb{R}^{N \times O}$, and a standard deviation $\sigma \in \mathbb{R}^{N \times O}$, which together parameterize a Gaussian distribution over actions for each of the prototypes. Given the encoded state z_s , the actor computes the similarity scores between the input and each prototype $\text{sim}(z_s, p_i)$ using Equation 8. Then, the similarity scores are used to produce a weighted combination of the prototype Gaussians by summing over the means μ_i and standard deviations σ_i , proportionally to both the similarity values and the importance weights. In this way, more similar and more important prototypes have a greater influence on the final action distribution. Formally, the final action distribution parameters

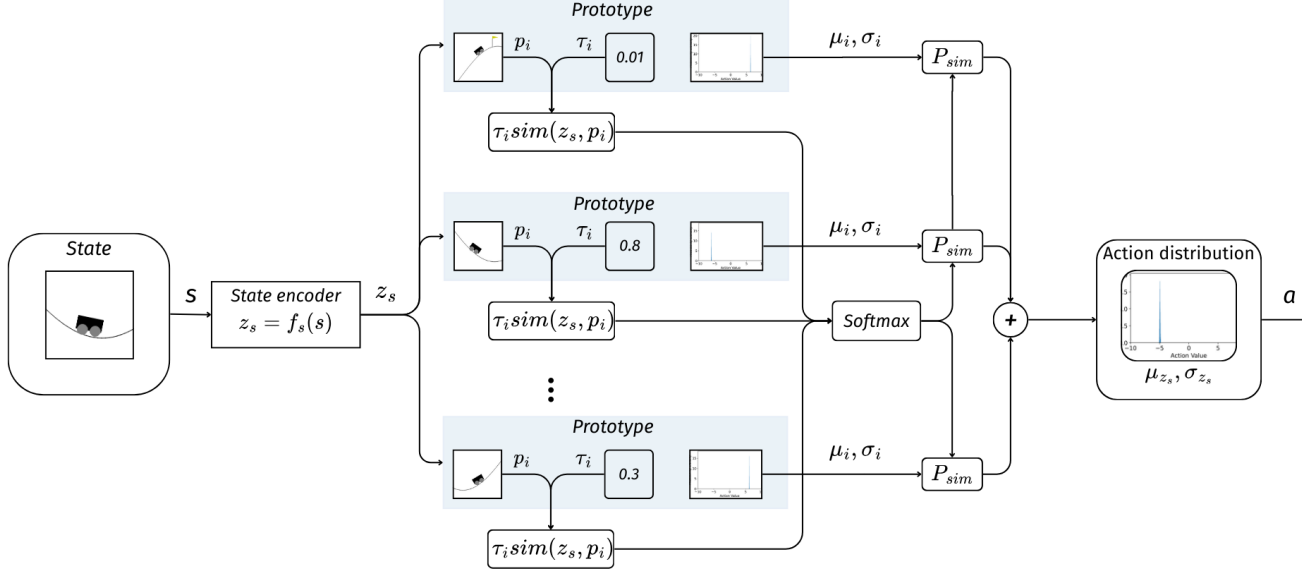


Figure 1. Overview of the *ProtoActor* architecture. The state is encoded through an encoder into a latent representation, which is passed to the *ProtoActor* to compute similarities with learnable prototypes. The weighted combination defines a Gaussian distribution for action sampling.

are computed as:

$$P_{sim_i} = \frac{\exp(\tau_i \cdot \text{sim}(z_s, p_i))}{\sum_{j=1}^N \exp(\tau_j \cdot \text{sim}(z_s, p_j))}, \quad i \in [1, \dots, N] \quad (10)$$

$$\mu_{z_s} = \sum_{i \in N} P_{sim_i} \mu_i, \quad \sigma_{z_s} = \sum_{i \in N} P_{sim_i} \sigma_i. \quad (11)$$

μ_{z_s} and σ_{z_s} define the parameters of the final Gaussian distribution from which the agent samples its action to be executed in the current state s during interaction with the environment. The remaining procedure follows the same steps described in the standard SAC framework.

Training. Training begins with the random initialization of the prototypes and their associated weights τ . Unlike standard SAC, *ProtoSAC* compares each encountered state to a fixed set of prototypes; As a result, the agent effectively interacts with a compressed view of the state space, which may limit the diversity of its experience. To mitigate this effect, *ProtoSAC* periodically updates its prototypes to better reflect underexplored regions of the environment. Every M episodes, we identify the prototypes whose importance weight τ falls below the q quantile across recent interactions (in our implementation, q is set to 0.5 after empirical evaluation). These prototypes are considered under-performing or underutilized and are selected for replacement. New prototype candidates p_{c_i} are sampled from the replay buffer by selecting states s_i that are least similar to the current prototype set:

$$p_{c_i} = \underset{z_{s_i}}{\operatorname{argmax}} \left(\frac{1}{\sqrt{\sum_{n \in N} \text{sim}(z_{s_i}, p_n) + \epsilon}} \right), \quad z_{s_i} = f_s(s_i) \quad (12)$$

This encourages exploration of novel areas in the state space and helps avoid prototype redundancy. For each newly selected prototype, we compute the mean and standard deviation of the action distribution that would have been assigned had the state been encountered during normal training, following Equation 10. These parameters are then assigned to the prototype, along with a randomly initialized similarity weight τ . The full prototype set is then updated accordingly. To improve prototype selection and ensure effective coverage of the state space, we introduce three supplementary loss terms to the actor’s objective:

- a **τ Regularization Loss** ($\mathcal{L}_\tau$) encourages the model to use the least number of prototypes by penalizing the importance weights τ_i . This ensures that only a small portion of prototypes are used, therefore leading to simple explanations:

$$\mathcal{L}_\tau = \sum_{i \in N} \tau_i; \quad (13)$$

- an **Orthogonality Loss** ($\mathcal{L}_{orth}$) enforces approximate orthogonality between prototypes. This ensures that they cover distinct directions in latent space rather than collapsing onto redundant regions:

$$\mathcal{L}_{orth} = \|PP^\top - I\|_F, \quad (14)$$

where $\|\cdot\|_F$ denotes the Frobenius norm and I is the identity matrix.

- a **Negative Entropy Loss** ($\mathcal{L}_{ent}$) encourages each state to activate more than one prototype with significant

similarity, avoiding overly “hard” assignments. If only a single prototype dominates, interpretability is weakened: the agent appears to always “copy” one prototype, rather than reasoning over multiple case-based examples:

$$\mathcal{L}_{ent} = \sum_{n \in N} P_{sim} \log(P_{sim}), \quad (15)$$

where $P_{sim} \in \mathbb{R}^N$ denotes the softmax-normalized similarity distribution over prototypes for a given state, defined in Equation 10.

The final loss of *ProtoSAC* can be written as:

$$\mathcal{L} = \mathcal{L}_{\pi} + \alpha \mathcal{L}_{ent} + \gamma \mathcal{L}_{orth} + \gamma \mathcal{L}_{\tau} \quad (16)$$

where $\mathcal{L}_{\pi}$ is the original SAC actor loss (defined in Equation 6), α is the entropy coefficient, and $\gamma = 0.0001$.

5. Results

In this section, we evaluate whether *ProtoSAC* can achieve performance comparable to the standard SAC algorithm and state-of-the-art self-explainable DRL method, Shared-PW-Net. Additionally, we present and analyze key prototypes learned during training to better understand how the model represents different aspects of the task. Finally, we evaluate the robustness and the faithfulness of the learned prototypes.

Experiment Setup. We test our model on 8 popular benchmark environments with increasing complexity: in Pendulum-v1, the agent’s objective is to swing and balance a pendulum in the upright position by applying continuous torque; in MountainCarContinuous-v0, the agent must drive a car up a steep hill; in MuJoCo InvertedPendulum-v5, the objective is to balance a pole upright on a moving cart; in LunarLanderContinuous-v3, the agent controls a lunar lander and must perform a soft landing on a target area; in MuJoCo HalfCheetah-v5, the agent controls a planar cheetah robot to run as fast as possible; in MuJoCo Hopper-v5, the agent must make a 2D one-legged robot hop forward without falling; in MuJoCo Humanoid-v5, the agent controls a high-dimensional biped to walk and stay upright; and in CarRacing-v3, the agent drives a car on a procedurally generated racetrack and must complete laps while staying on the track. In the first four environments, the optimal behavior is well known, which facilitates clearer inspection and evaluation of the learned policies through our prototypical explanations; the others represent instead complex tasks that are harder to learn.

For the experiments, we use the Stable-Baselines3 library, and we implement our model to fit within it. Our hyperparameter selection is based on the recommended val-

ues available in the official GitHub repository³. Following the recommended configurations, in the case of the MountainCarContinuous-v0 environment, we integrate generalized State-Dependent Exploration (gSDE) (Raffin et al., 2022) in place of standard action noise to better handle exploration in continuous action spaces. We use a total of 30 prototypes for each model, except for the Inverted Pendulum, where we use 60 prototypes instead. The prototype set is updated every 20 episodes. These values were selected via grid search over the range and represent a safe upper bound: we intentionally initialize more prototypes than expected to be needed. The regularization loss $\mathcal{L}_{\tau}$ then drives the model to concentrate explanatory weight on the minimal sufficient subset, rendering excess prototypes functionally inactive. The full set of hyperparameters used for each environment and ablation studies over losses and prototypes are reported in the supplementary material.

Performances. We now compare the performances of our model with those of SAC and another self-explainable model, Shared-PW-Net. As shown in Table 1, Shared-PW-Net is the only existing prototype-based approach that allows training self-explainable agents in continuous action spaces without requiring manual prototype selection. In Table 2, we report the cumulative rewards for *ProtoSAC*, Shared-PW-Net, and SAC, evaluated on 30 simulations. The results indicate that *ProtoSAC* demonstrates consistently strong performance across all the environments. We assess statistical significance of pairwise comparisons via Welch’s *t*-tests with Cohen’s *d* effect sizes across the 30 evaluation runs; full results are reported in Appendix C (Table 6). In the Pendulum environment, *ProtoSAC* outperforms Shared-PW-Net by a significant margin and performs competitively with SAC, even though with a higher variance. For LunarLander, *ProtoSAC* achieves the highest performance with the lowest variance, marking a significant improvement over Shared-PW-Net, though the difference with SAC does not reach statistical significance. In the MountainCar environment, while Shared-PW-Net attains the highest mean score, all methods operate near the task’s performance ceiling. Despite reaching statistical significance, the absolute differences are small in magnitude and the task is effectively solved by all three methods. Notably, in the InvertedPendulum environment, *ProtoSAC* matches SAC’s perfect score, while Shared-PW-Net fails to achieve meaningful performance. This is mainly due to the fact that while the training of Shared-PW-Net is independent from the task, as it is only trained to replicate the behavior of the black-box, *ProtoSAC* is instead directly trained to solve the specific task, thus providing stronger performances in control tasks such as Pendulum and InvertedPendulum. In high-dimensional con-

³<https://github.com/DLR-RM/rl-baselines3-zoo/blob/master/hyperparams/sac.yml>

Table 2. Comparison of prototype-based approaches for explainable DRL. We report the mean and standard deviation of rewards over 30 simulations. Best values are highlighted in bold, while values within one standard deviation of the best values are underlined.

Environment	SAC	Shared-PW-Net	ProtoSAC
Pendulum	-141.79 ± 61.79	-524.33 ± 386.99	<u>-152.24 ± 83.50</u>
Lunar Lander	256.59 ± 88.26	258.77 ± 55.17	282.42 ± 19.90
Mountain Car	97.55 ± 0.27	97.63 ± 0.30	95.29 ± 0.77
Inverted Pendulum	1000.0 ± 0.0	33.43 ± 10.37	1000.0 ± 0.0
Hopper	3380.251 ± 1.980	3374.412 ± 3.049	3386.222 ± 8.855
HalfCheetah	11098.333 ± 150.083	10369.168 ± 1248.026	9875.183 ± 77.872
Humanoid	4574.069 ± 1358.091	496.586 ± 133.035	4953.814 ± 3.268
CarRacing	232.968 ± 143.422	-32.528 ± 9.124	369.047 ± 189.355

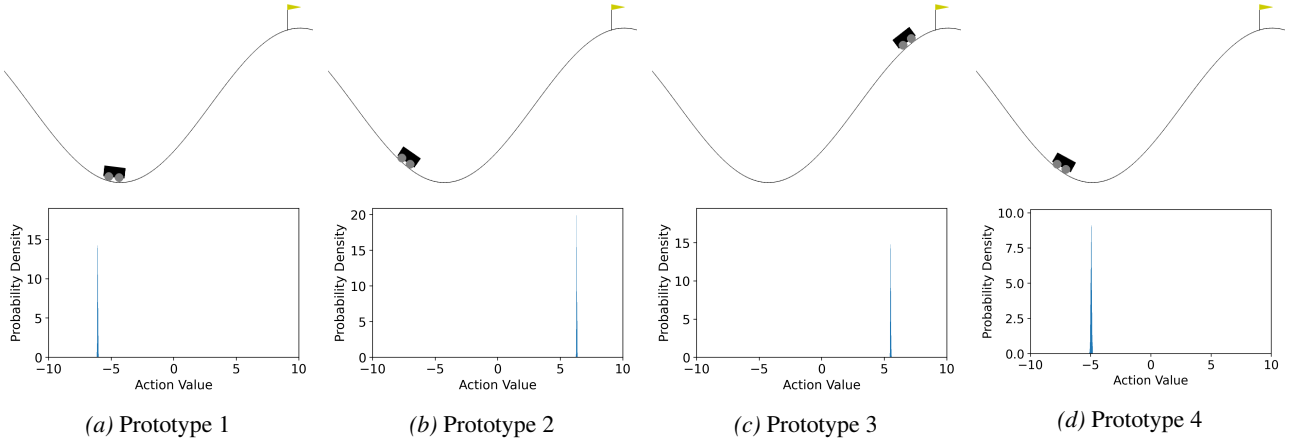


Figure 2. Examples of the 4 most important prototypes according to τ in the MountainCar environment. Each prototype is represented by its associated state and the action Gaussian distribution. A negative action is associated with a leftward motion (Figures a and d), while a positive action corresponds to a rightward movement (Figures b and c).

tinuous control environments, the results further confirm the robustness of *ProtoSAC*. In the Hopper task, *ProtoSAC* achieves the highest cumulative reward, significantly outperforming both SAC and Shared-PW-Net. On HalfCheetah, SAC remains the best-performing model with a large performance gap. This environment requires high-frequency torque control, where the optimal policy is unlikely to be well-approximated by a small set of prototype-based clusters. *ProtoSAC* still maintains competitive results compared to Shared-PW-Net, suggesting the performance gap is specific to comparison with unconstrained SAC rather than reflecting a general failure of the approach. On Humanoid and CarRacing, *ProtoSAC* achieves the highest cumulative reward in both environments, significantly outperforming Shared-PW-Net. The improvement over SAC is statistically significant in CarRacing, while the difference in Humanoid does not reach significance, indicating comparable performance to the unconstrained baseline.

Prototypes. To understand how the agent’s policy interacts with the environment and how the model captures the most relevant parts of the state space, we visualize the 4

prototypes with the highest τ scores. These represent the most influential components in the policy’s decision-making process. Each prototype consists of a representative state (visualized as an image) and an associated action distribution, modeled as a Gaussian over the action space. Figure 2 shows the most important prototypes according to τ for the MountainCarContinuous environment. In this task, the agent must build momentum through oscillating in order to reach the hilltop. Several prototypes, such as Figures 2a and 2d, are associated with leftward motion (i.e., negative actions), aimed at gathering momentum. Others, including Figures 2b and 2c, correspond to rightward thrusts (i.e., positive actions) intended to exploit the momentum and reach the goal. Figure 3 presents four of the most important prototypes for the LunarLander environment. Here, each action is a two-dimensional vector composed of the main engine throttle and lateral booster control. Consequently, the action distribution for each prototype is a two-dimensional Gaussian. The visualized prototypes illustrate how the agent has learned to perform fine-grained control in a highly dynamic setting. For example, Figures 3c and 3d show how the model balances the main engine and lateral boosters to

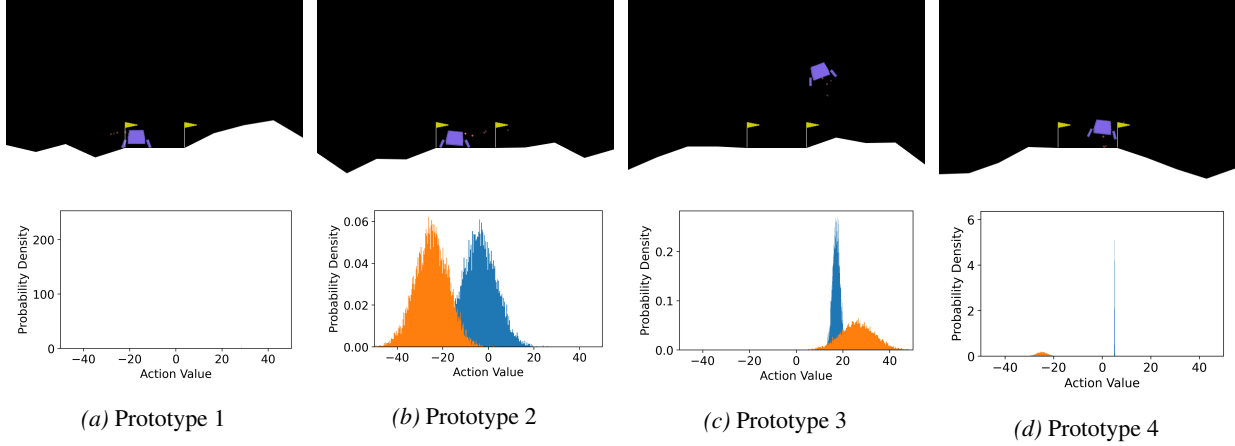


Figure 3. Examples of 4 most important prototypes according to τ in the LunarLander environment. Each prototype is illustrated with its corresponding state and action, with a Gaussian distribution for each action component. The first component (blue) controls the main engine throttle, while the second (orange) controls the throttle of the lateral boosters.

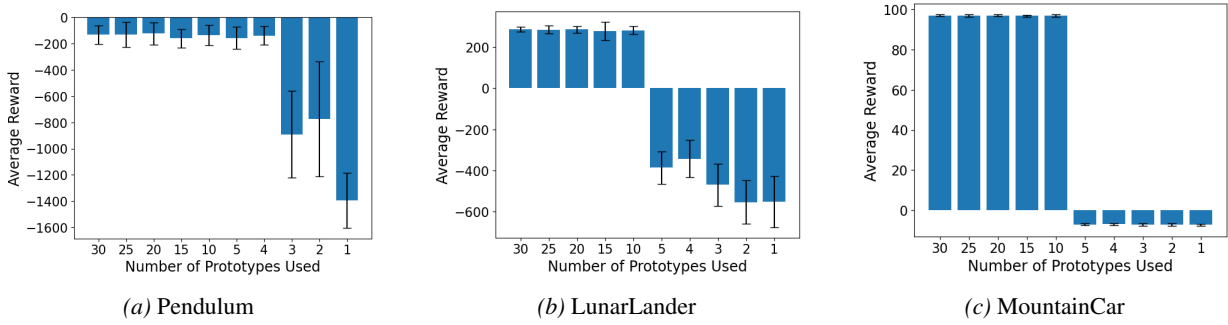


Figure 4. Sensitivity analysis with different number of prototypes. We remove from each pretrained model the least used prototypes according to τ and evaluate the resulting performance.

align with the target. In contrast, Figures 3a and 3b illustrate two examples of landings: the first corresponds to a state in which the agent remains stationary, suggesting it has learned to recognize when inaction is optimal; the second shows a landing that requires balancing between the boosters and the main engine to avoid crashing. Here, it is important to highlight that while the shuttle position is similar on the map, the difference lies in its current speed, indicated by the red sparks, and also quantitatively measurable when running the environment. Overall, these examples highlight the model’s ability to represent diverse behaviors and adapt its policy to the complexity of the task.

Sensitivity Analysis. Here, we assess the model’s capacity to retain only meaningful prototypes to ensure sparse and simple explanations. We therefore study the impact of each prototype on performance by progressively removing the least-used prototypes according to the importance weight τ . In Figure 4, we report the results of this analysis for the Pendulum, LunarLander, and MountainCar environments. In all three environments, reward remains stable even when several prototypes are removed, only declining noticeably

after a considerable number are eliminated. In particular, in the simple task of Pendulum, 5 prototypes are sufficient to retain the performances of the full model, whereas in more complex tasks such as LunarLander, the amount of needed performances rises to 10. This observation suggests that the agent is capable of using τ to select a small subset of the most important prototypes. This makes it sufficient to only inspect the prototypes with the highest associated τ values to capture and explain the behavior of the agent, and it is not necessary to examine all prototypes to understand its decision-making.

Explanation Evaluation. We evaluate the faithfulness of the explanations by measuring, for each model, the mean squared deviation between the original action and the action recomputed after removing the most active prototype. The results, reported in Table 3, show that *ProtoSAC* consistently exhibits larger deviations than *Shared-PW-Net* across all tasks, indicating a stronger functional dependency of the policy on the most active prototype. In Hopper and HalfCheetah, the deviation almost doubles for *ProtoSAC*, suggesting that its prototypes encode more decisive control

Table 3. Explanation fidelity evaluation. Mean squared deviation ($\pm$ std) between original action and action after removing the most active prototype.

Environment	Shared-PW-Net	ProtoSAC
Hopper	0.182 ± 0.174	0.534 ± 0.354
HalfCheetah	0.402 ± 0.282	0.696 ± 0.528
Humanoid	0.050 ± 0.044	0.084 ± 0.021
CarRacing	0.188 ± 0.064	0.383 ± 0.525

patterns compared to those of Shared-PW-Net. A similar trend is observed in Humanoid and CarRacing, where *ProtoSAC* again yields higher deviations, which implies that its explanations are more tightly coupled to the underlying policy behavior.

6. Conclusions

In this work, we introduced *ProtoSAC*, a novel self-interpretable reinforcement learning architecture that integrates prototype-based reasoning directly into the SAC framework. Unlike previous approaches that rely on post-hoc explanations or imitation learning, *ProtoSAC* generates interpretable policies from scratch. By representing policies as similarity-weighted combinations over learned prototypes, our method enables decomposable and prototype-based decision-making. Through experiments on continuous action-space environments, we demonstrated that *ProtoSAC* achieves competitive performance compared to SAC, while outperforming state-of-the-art self-explainable models and offering more faithful explanations.

While our approach improves the transparency of DRL agents, this design choice comes with certain trade-offs. The additional complexity can lead to slower convergence and higher variance during early training phases compared to standard SAC. Additionally, the quality of interpretability depends on the selection and diversity of learned prototypes: careful initialization and update strategies are essential for maintaining both performance and explainability. We address these challenges through prototype replacement mechanisms and dedicated regularization losses, which help stabilize training while preserving competitive performance. Looking forward, several promising directions could further enhance *ProtoSAC*’s efficiency and interpretability. More sophisticated initialization schemes, such as curriculum learning or meta-learning approaches, could accelerate convergence and reduce variance. Integrating complementary interpretability mechanisms, such as symbolic reasoning or concept bottlenecks, may enable richer, multi-level explanations that capture both low-level action decisions and high-level strategic reasoning. Additionally, while *ProtoSAC* is designed for continuous action spaces, discrete variants of SAC have been proposed in the literature (Christodoulou,

2019), suggesting that *ProtoSAC* could in principle be extended to discrete settings. These extensions could broaden the applicability of prototype-based interpretable RL to more complex domains while maintaining the inspectability that is central to our approach.

Acknowledgements

This work was supported by French state aid managed by the National Research Agency under the France 2030 program, with the reference “PANDORA ANR-24-CE23-0950”.

Impact Statement

This paper introduces a novel self-interpretable DRL model that achieves state-of-the-art performance in continuous control tasks, compared to other self-interpretable models. This advancement represents an important step toward bridging high-performance DRL and safety-critical applications, where more inspectable decision-making is valuable.

References

- Bastani, O., Pu, Y., and Solar-Lezama, A. Verifiable reinforcement learning via policy extraction. *Advances in neural information processing systems*, 31, 2018.
- Borzillo, C., Ragno, A., Capobianco, R., et al. Understanding deep rl agent decisions: a novel interpretable approach with trainable prototypes. In *CEUR Workshop Proceedings Vol-3518*, 2023.
- Chen, C., Li, O., Tao, D., Barnett, A., Rudin, C., and Su, J. K. This looks like that: deep learning for interpretable image recognition. *Advances in neural information processing systems*, 32, 2019.
- Christodoulou, P. Soft actor-critic for discrete action settings. *arXiv preprint arXiv:1910.07207*, 2019.
- Delfosse, Q., Sztwiertnia, S., Rothermel, M., Stammer, W., and Kersting, K. Interpretable concept bottlenecks to align reinforcement learning agents. *Advances in Neural Information Processing Systems*, 37:66826–66855, 2024.
- Došilović, F. K., Brčić, M., and Hlupić, N. Explainable artificial intelligence: A survey. In *2018 41st International convention on information and communication technology, electronics and microelectronics (MIPRO)*, pp. 0210–0215. IEEE, 2018.
- Frosst, N. and Hinton, G. Distilling a neural network into a soft decision tree. 2018.
- Glanois, C., Weng, P., Zimmer, M., Li, D., Yang, T., Hao, J., and Liu, W. A survey on interpretable reinforcement learning. *Machine Learning*, pp. 1–44, 2024.

- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pp. 1861–1870. Pmlr, 2018.
- Kenny, E. M., Tucker, M., and Shah, J. Towards interpretable deep reinforcement learning with human-friendly prototypes. In *The Eleventh International Conference on Learning Representations*, 2023.
- Kiumarsi, B., Vamvoudakis, K. G., Modares, H., and Lewis, F. L. Optimal and autonomous control using reinforcement learning: A survey. *IEEE transactions on neural networks and learning systems*, 29(6):2042–2062, 2017.
- Kober, J., Bagnell, J. A., and Peters, J. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013.
- Koh, P. W., Nguyen, T., Tang, Y. S., Musmann, S., Pierson, E., Kim, B., and Liang, P. Concept bottleneck models. In III, H. D. and Singh, A. (eds.), *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 5338–5348. PMLR, 13–18 Jul 2020.
- Milani, S., Topin, N., Veloso, M., and Fang, F. Explainable reinforcement learning: A survey and comparative review. *ACM Comput. Surv.*, 56(7), April 2024. ISSN 0360-0300. doi: 10.1145/3616864. URL <https://doi.org/10.1145/3616864>.
- Qing, Y., Liu, S., Song, J., Wang, H., and Song, M. A survey on explainable reinforcement learning: Concepts, algorithms, challenges, 2023. URL <https://arxiv.org/abs/2211.06665>.
- Raffin, A., Kober, J., and Stulp, F. Smooth exploration for robotic reinforcement learning. In *Conference on robot learning*, pp. 1634–1644. PMLR, 2022.
- Ragodos, R., Wang, T., Lin, Q., and Zhou, X. Explaining a reinforcement learning agent via prototyping. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=nyBJcnhjAoy>.
- Rudin, C. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5): 206–215, May 2019. ISSN 2522-5839. doi: 10.1038/s42256-019-0048-x. URL <http://dx.doi.org/10.1038/s42256-019-0048-x>.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.
- Verma, A., Murali, V., Singh, R., Kohli, P., and Chaudhuri, S. Programmatically interpretable reinforcement learning. In *International conference on machine learning*, pp. 5045–5054. PMLR, 2018.
- Wang, B., Li, L., Nakashima, Y., and Nagahara, H. Learning bottleneck concepts in image classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10962–10971, 2023a.
- Wang, J., Liu, H., and Jing, L. Transparent embedding space for interpretable image recognition. *IEEE Transactions on Circuits and Systems for Video Technology*, 2023b.
- Yarats, D., Fergus, R., Lazaric, A., and Pinto, L. Reinforcement learning with prototypical representations. In *International Conference on Machine Learning*, pp. 11920–11931. PMLR, 2021.

A. Training Algorithm

Algorithm 1 ProtoSAC Training

- 1: **Require:** Environment E , number of prototypes N , update interval M , quantile threshold q , loss coefficients α, γ
- 2: Initialize replay buffer $\mathcal{D}$
- 3: Initialize critic networks Q_{ϕ_1}, Q_{ϕ_2} and target networks $Q_{\bar{\phi}_1}, Q_{\bar{\phi}_2}$
- 4: Initialize encoder f_s , prototype matrix $P \in \mathbb{R}^{N \times H}$, importance weights $\tau \in \mathbb{R}^N$, means $\mu \in \mathbb{R}^{N \times O}$, standard deviations $\sigma \in \mathbb{R}^{N \times O}$
- 5: **for** each episode **do**
- 6: **for** each environment step **do**
- 7: Observe state s and encode: $z_s \leftarrow f_s(s)$
- 8: Compute similarities via Eq. (8):

$$\text{sim}(z_s, p_i) = \log \left(\frac{\|z_s - p_i\|_2 + 1}{\|z_s - p_i\|_2 + \epsilon} \right)$$

- 9: Compute importance-weighted distribution via Eq. (10):

$$P_{\text{sim}_i} = \frac{\exp(\tau_i \cdot \text{sim}(z_s, p_i))}{\sum_{j=1}^N \exp(\tau_j \cdot \text{sim}(z_s, p_j))}, \quad i \in [1, \dots, N]$$

- 10: Compute action distribution parameters via Eq. (11):

$$\mu_{z_s} = \sum_{i=1}^N P_{\text{sim}_i} \mu_i, \quad \sigma_{z_s} = \sum_{i=1}^N P_{\text{sim}_i} \sigma_i$$

- 11: Sample action $a \sim \mathcal{N}(\mu_{z_s}, \sigma_{z_s})$
- 12: Execute a , observe reward r and next state s' ; store (s, a, r, s') in $\mathcal{D}$
- 13: **end for**
- 14: **for** each gradient step **do**
- 15: Sample minibatch $(s, a, r, s') \sim \mathcal{D}$
- 16: Update critics: minimize $\mathcal{L}_Q(\phi_i), i = 1, 2$ via Eq. (5)
- 17: Update actor: minimize full loss $\mathcal{L}$ via Eq. (16)
- 18: Update target networks: $\bar{\phi}_i \leftarrow \rho \bar{\phi}_i + (1 - \rho) \phi_i$
- 19: **end for**
- 20: **if** episode mod $M = 0$ **then**
- 21: Identify underutilized prototypes: $U = \{i : \tau_i < \text{quantile}(\tau, q)\}$
- 22: **for** each $i \in U$ **do**
- 23: Select replacement candidate via Eq. (12):

$$p_i \leftarrow \arg \max_{z_{s_j}} \frac{1}{\sqrt{\sum_n \text{sim}(z_{s_j}, p_n) + \epsilon}}$$

- 24: Initialize (μ_i, σ_i) from current policy via Eq. (10)
 - 25: Reinitialize τ_i randomly
 - 26: **end for**
 - 27: **end if**
 - 28: **end for**
-

B. Experimental Settings and Reproducibility

In Table 4 we detail the hyperparameters used in the experiments. We perform our experiments on a machine equipped with a Intel(R) Xeon(R) CPU E5-2698 v4 @ 2.20GHz and a Tesla V100-SXM2-32GB. In Table 5 we report the average training times (in seconds) for *ProtoSAC* and *SAC*.

Table 4. Training hyperparameters for each environment.

Hyperparameter	MountainCarContinuous-v0	Pendulum-v1	Mujoco Environments	LunarLanderContinuous-v3	CarRacing-v3
Timesteps	400,000	1,000,000	1,000,000	1,000,000	1,000,000
Learning Rate	3e-4	1e-3	3e-4	7.3e-4	3e-4
Buffer Size	50,000	1,000,000	1,000,000	1,000,000	300000
Batch Size	512	256	256	256	256
Entropy Coefficient	0.1	auto	auto	auto	auto
Train Frequency	32	1	1	1	8
Gradient Steps	32	1	1	1	8
Gamma	0.9999	0.99	0.99	0.99	0.99
Tau	0.01	0.005	0.005	0.01	0.02
Learning Starts	0	200	10,000	10,000	1000
Use SDE	True	-	-	-	-

Table 5. Average training times (in minutes) for SAC and ProtoSAC across the environments.

Environment	SAC (min)	ProtoSAC (min)
Pendulum-v1	314	490
LunarLanderContinuous-v3	378	540
MountainCarContinuous-v0	157	204
InvertedPendulum-v5	314	426
Hopper-v5	210	349
HalfCheetah-v5	317	248
Humanoid-v5	1240	3180
CarRacing-v3	462	569

C. Statistical Significance Analysis

To assess the statistical significance of the performance differences reported in Table 2, we perform the Welch’s t -tests and compute Cohen’s d effect sizes for all pairwise comparisons, using the 30 evaluation runs already reported. Welch’s t -test is chosen as it does not assume equal variances across methods. We note that the normality assumption underlying the test should be verified in practice; given the large sample size ($n = 30$), the Central Limit Theorem provides reasonable justification. To account for multiple comparisons (15 tests), we report both raw p -values and Bonferroni-corrected significance thresholds ($\alpha = 0.05/15 \approx 0.0033$).

Overall, ProtoSAC significantly outperforms Shared-PW-Net across nearly all environments, with large effect sizes ($d > 1$ in Pendulum, Hopper, Humanoid, and CarRacing). Comparisons against SAC are more mixed: ProtoSAC matches SAC on Pendulum, LunarLander, and Humanoid (all ns), outperforms it on Hopper and CarRacing, but underperforms on MountainCar and HalfCheetah.

The large negative effect size in HalfCheetah ($d = -10.23$ vs. SAC) warrants further discussion. This environment requires high-speed locomotion with fine-grained torque control, where the optimal policy likely demands a rich, high-frequency mapping from states to actions. Prototype-based compression may be insufficient to capture such complexity, as the state representation is not easily partitioned into a small set of archetypal behaviors. This suggests that ProtoSAC, in its current form, is best suited to environments where the optimal policy is *behaviorally compressible*—i.e., where a small number of representative state clusters suffices to define effective control strategies. Addressing this limitation, for instance through adaptive prototype counts or hierarchical prototype structures, remains an important direction for future work.

Table 6. Pairwise statistical comparisons between ProtoSAC and baselines. Significance levels: $*p < 0.05$, $**p < 0.01$, $***p < 0.001$, ns = not significant. InvertedPendulum: both SAC and ProtoSAC achieve a perfect score of 1000 ± 0.0 across all 30 runs; a t -test is undefined.

Environment	Comparison	Mean Diff	t -stat	p -value	Cohen’s d
Pendulum	ProtoSAC vs SAC	−10.45	−0.55	0.584 (ns)	−0.14
Pendulum	ProtoSAC vs S-PW-Net	+372.09	5.15	< 0.001***	1.33
LunarLander	ProtoSAC vs SAC	+25.83	1.56	0.128 (ns)	0.40
LunarLander	ProtoSAC vs S-PW-Net	+23.65	2.21	0.034*	0.57
MountainCar	ProtoSAC vs SAC	−2.26	−15.17	< 0.001***	−3.92
MountainCar	ProtoSAC vs S-PW-Net	−2.34	−15.51	< 0.001***	−4.01
InvPendulum	ProtoSAC vs SAC	0.00	—	—	0.00
Hopper	ProtoSAC vs SAC	+5.97	3.60	0.001**	0.93
Hopper	ProtoSAC vs S-PW-Net	+11.81	6.91	< 0.001***	1.78
HalfCheetah	ProtoSAC vs SAC	−1223.15	−39.62	< 0.001***	−10.23
HalfCheetah	ProtoSAC vs S-PW-Net	−493.99	−2.16	0.039*	−0.56
Humanoid	ProtoSAC vs SAC	+379.75	1.53	0.137 (ns)	0.40
Humanoid	ProtoSAC vs S-PW-Net	+4457.23	183.46	< 0.001***	47.37
CarRacing	ProtoSAC vs SAC	+136.08	3.14	0.003**	0.81
CarRacing	ProtoSAC vs S-PW-Net	+401.58	11.60	< 0.001***	3.00

D. Training Dynamics

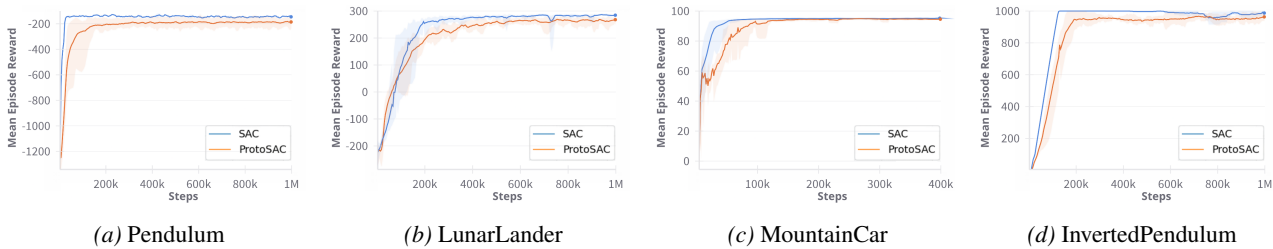


Figure 5. Performance comparison between *ProtoSAC* (orange) and *SAC* (blue), showing episode rewards over the course of training for each tested environment. Shaded areas represent the standard deviation across five runs.

Here, we evaluate the impact of the prototype-based architecture on the SAC algorithm. Figure 5 presents the training dynamics of *ProtoSAC* compared to the *SAC* baseline across the first four continuous control environments. Training performance is measured using the average cumulative reward evaluated over 5 independent runs. We observe that *ProtoSAC* achieves comparable performance to the baseline, although it generally requires a slight amount of additional training steps to converge. While the peak performance reached by *ProtoSAC* is slightly lower in some environments, the gap is minimal. Given that *ProtoSAC* additionally provides interpretability throughout the learning process, this trade-off is arguably favorable. An additional observation is the increased variability in performance across training runs for *ProtoSAC* compared to the *SAC* baseline. This variability is more pronounced during the early stages of training but tends to diminish as the model stabilizes. This higher variance is likely due to the method’s sensitivity to prototype initialization: when prototypes are not sufficiently representative, they can interfere with optimal learning and result in suboptimal performance. We observe that in simpler environments such as *MountainCar*, the performance gap between *ProtoSAC* and the baseline is negligible. However, as the environment becomes more complex, as in *LunarLander*, the gap increases slightly. Nevertheless, *ProtoSAC* still achieves performance that remains comparable to the baseline.

E. Ablation Study

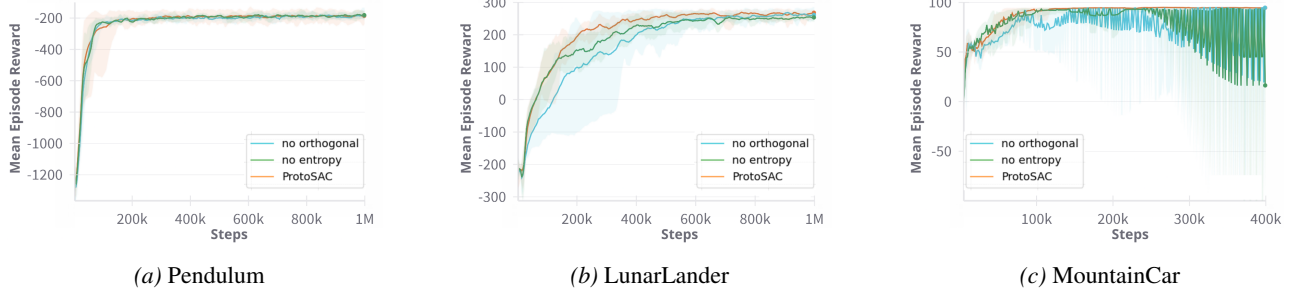


Figure 6. The ablation study in the continuous environment. We can see that except for the Pendulum task, removing either the orthogonal loss (blue) or the entropy loss (green) tends to introduce high instability during training, although in some cases the final performance remains competitive.

To assess the contribution of each component of our method, we conduct an ablation study by systematically removing individual loss terms from the training process. Specifically, we evaluate the impact of the orthogonal loss and the negative entropy loss on the training dynamics across all continuous control environments. As shown in Fig. 6, these losses play a crucial role in maintaining training stability, particularly in more complex environments. While in the Pendulum environment the differences are marginal, starting from LunarLander, we observe greater variability in the learning curves when either loss is removed.

In particular, removing the orthogonal loss leads to highly unstable behavior—sometimes achieving high rewards, but often resulting in poor performance. For MountainCar, the most noticeable effect is not on peak reward, but on how long the model can train before suffering from overfitting. Since we employ generalized State-Dependent Exploration (gSDE), which adjusts the policy’s standard deviation dynamically, excessive training can lead to saturation. In this setting, models trained with both auxiliary losses tend to delay overfitting, while those missing one of the losses experience instability sooner.

These findings suggest that the orthogonal loss and the negative entropy loss work in a complementary way: the orthogonal loss promotes diversity among prototypes, ensuring better coverage of the state space, while the negative entropy loss encourages the model to rely on more than one prototype with high similarity. Together, they help achieve more robust and generalized policies.

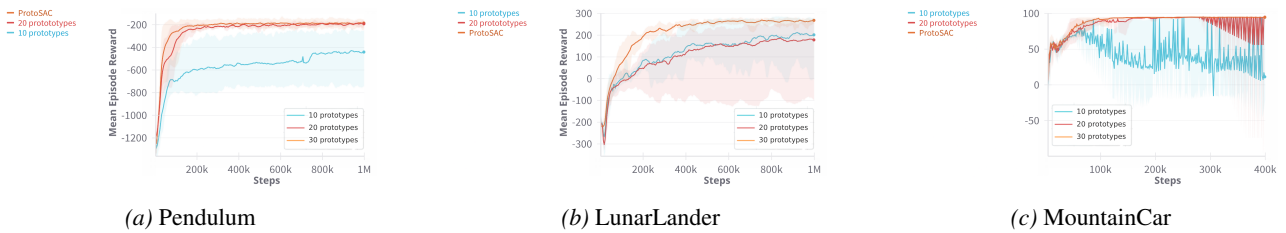


Figure 7. Evaluate the impact of prototype quantity on performance. We compare ProtoSAC using three different numbers of prototypes: 30 (orange), 20 (red), and 10 (blue), across the three continuous control environments.

To determine the appropriate number of prototypes for each environment, we conducted an ablation study by training *ProtoSAC* with three different prototype counts: 30, 20, and 10. The goal was to assess how the number of prototypes influences overall performance and training stability.

In the simplest environment, Pendulum, using only 10 prototypes is sufficient to solve the task, although the performance does not fully match the baseline in terms of average reward. With 20 prototypes, the performance slightly decreases compared to 30, likely due to the limited complexity of the environment, where fewer prototypes can still provide adequate coverage of the state space.

In contrast, in the more complex LunarLander environment, reducing the number of prototypes to 20 or 10 results in significantly less stable training and often leads to poor task completion. A similar trend is observed in MountainCar, here using 20 prototypes allows the agent to solve the task, but with increased variance and reduced robustness compared to the 30-prototype setup, while using 10 prototypes fails to provide sufficient coverage, resulting in unsuccessful task completion.

F. Pendulum Prototypes

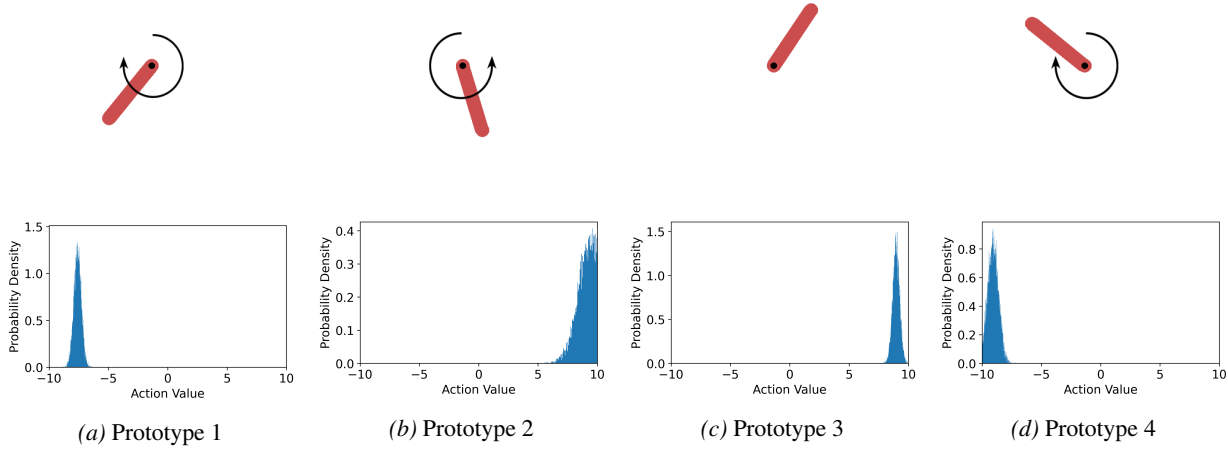


Figure 8. Examples of 4 most important prototypes according to τ in the Pendulum environment. Each prototype is represented by its associated state and the action Gaussian distribution.

In this section, we analyze the prototypes used in the Pendulum environment, as shown in Figure 8. Each prototype is linked to a specific state and an associated action, corresponding to the torque applied to the pendulum. As can be observed, the agent has correctly learned to apply a negative torque when the pendulum is in a leftward position and a positive torque when it is in a rightward position. Interestingly, the model has learned to concentrate most of its prototypes on the initial swing-up phase, rather than on states that allow the pendulum to maintain balance once already upright. This likely reflects the importance of the initial push in successfully completing the task.